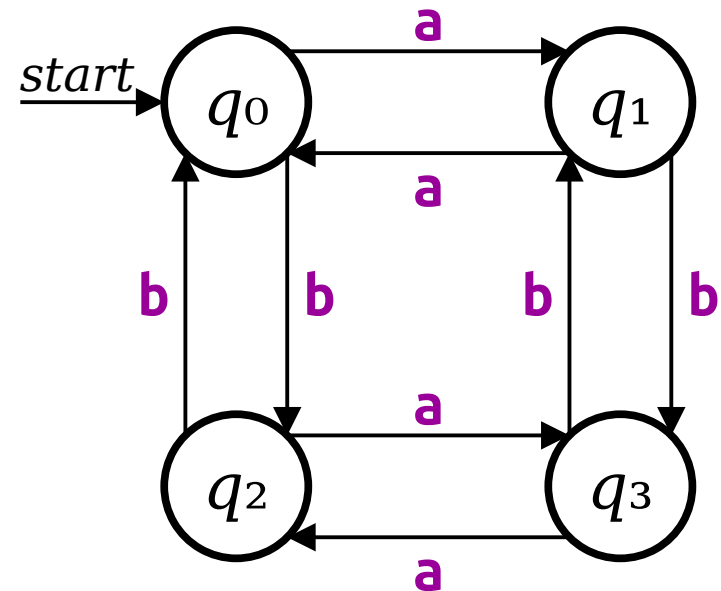


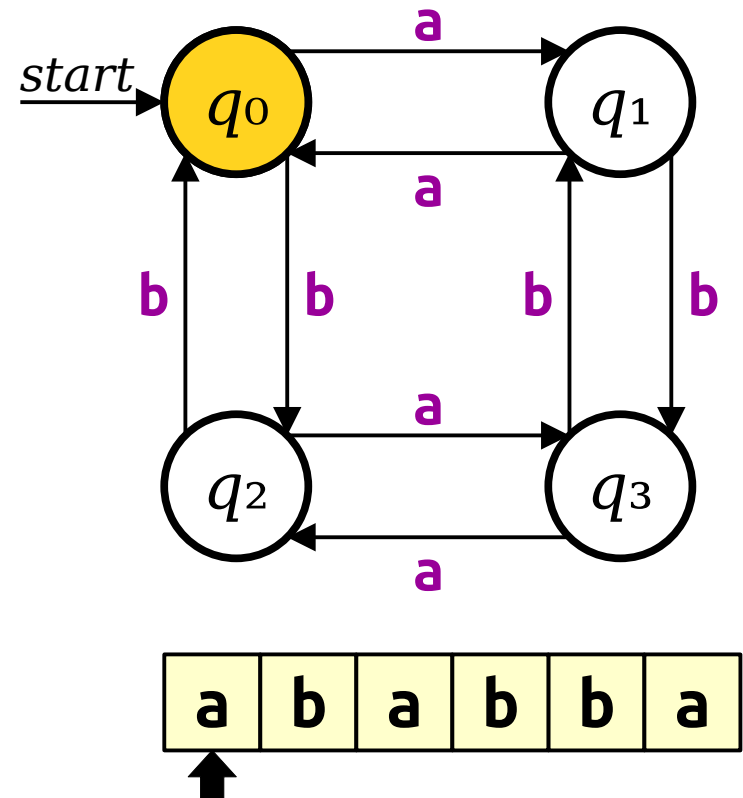
Modeling Finite Computation

- We will model a finite-memory computer as a collection of **states** linked by **transitions**.
- Each state corresponds to one possible configuration of the device's memory.
- Each transition indicates how memory changes in response to inputs.
- Some state is designated as the **start state**. The computation begins in that state.



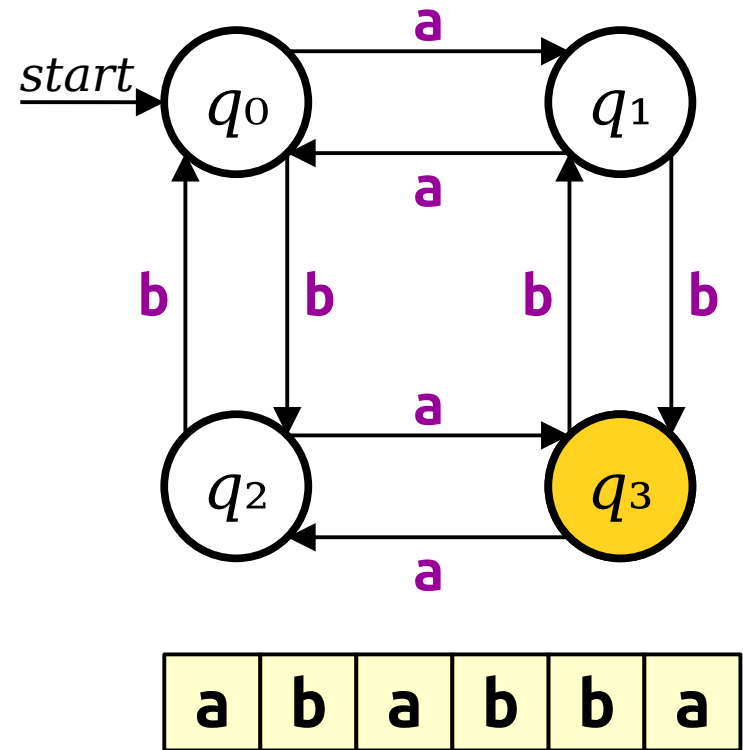
Modeling Finite Computation

- This device processes **strings** made of **characters**.
 - Each character represents some external input to the device.
 - The string represents the full sequence of inputs to the device.
- To run this device, we begin in our start state and scan the input from left to right.
- Each time the machine sees a character, it **changes state** by following the transition labeled with that character.



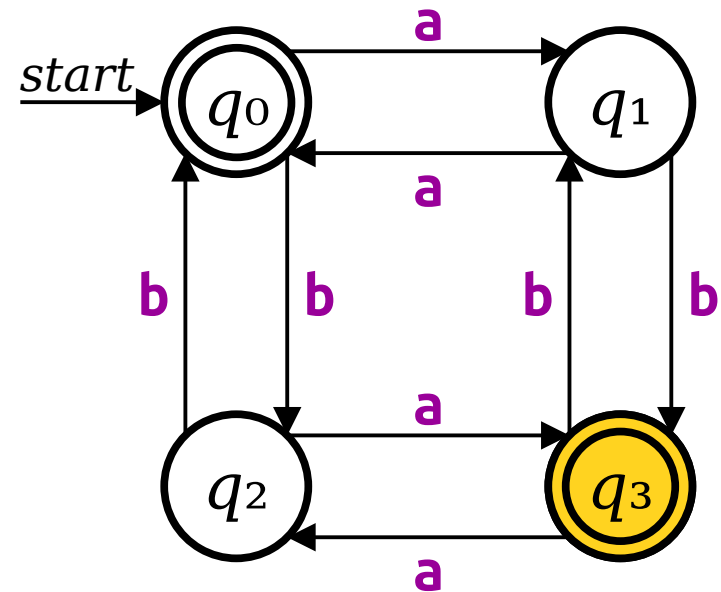
Modeling Finite Computation

- Once we've finished entering all the characters of our input, we need to obtain the result of the computation.
- In general, computers can produce all sorts of things as the result of a computation: a number, a piece of text, etc.
- As a simplifying assumption, we'll assume that we just need to get a single bit of output. That is, our machines will just say YES or NO.
- (This can be generalized – come talk to me after class if you're curious how!)



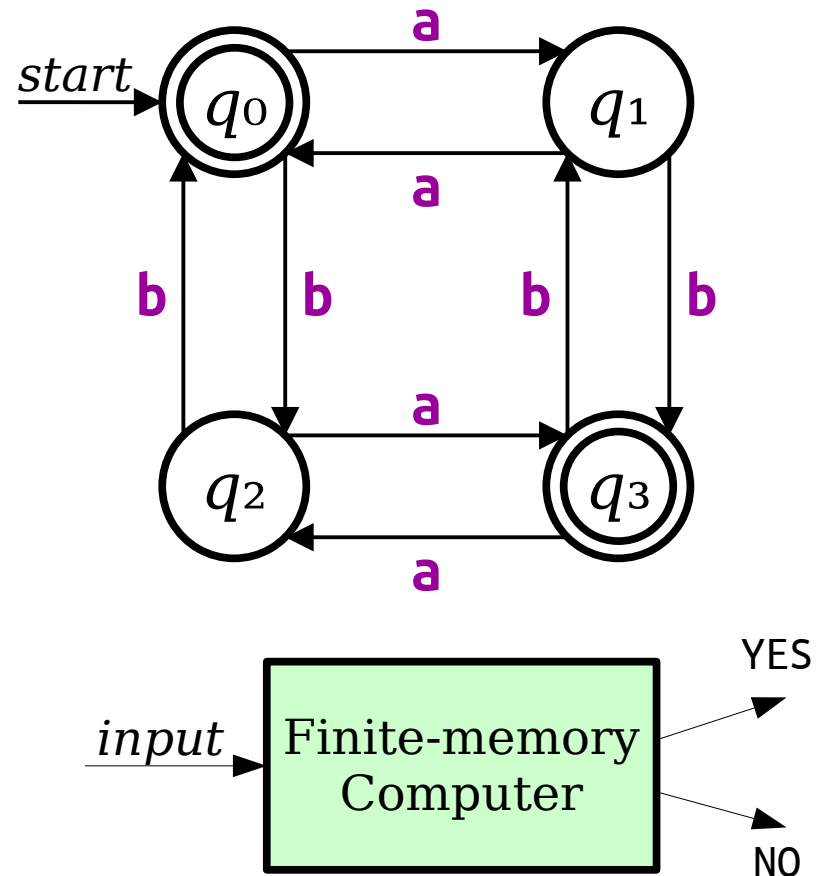
Modeling Finite Computation

- Some of the states in our computational device will be marked as **accepting states**. These are denoted with a double ring.
- If the device ends in an accepting state after seeing all the input, **accepts** the input (says YES)
- If the device does not end in an accepting state after seeing all the input, it **rejects** the input (says NO).

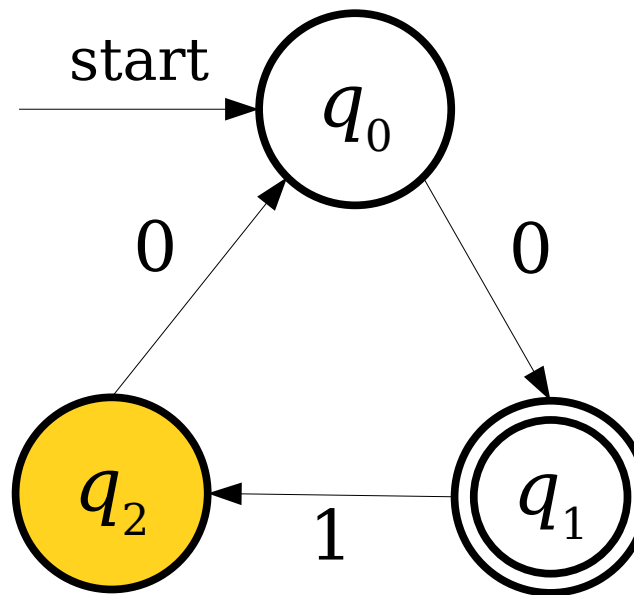


Finite Automata

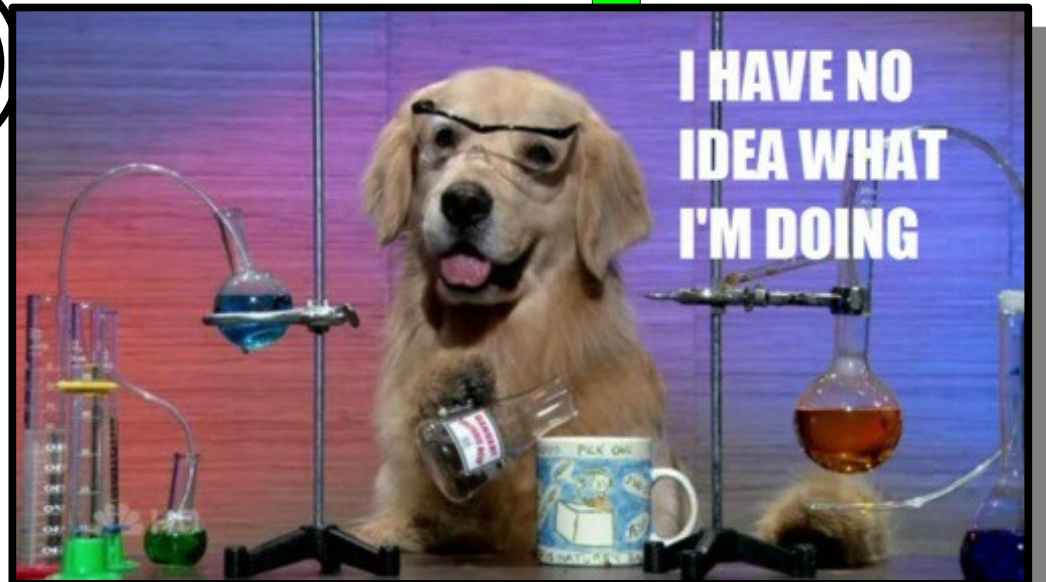
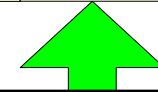
- This type of computational device is called a **finite automaton** (plural: **finite automata**).
- Finite automata model computers where (1) memory is finite and (2) the computation produces as YES/NO answer.
- In other words, finite automata model predicates, and do so with a fixed, finite amount of memory.



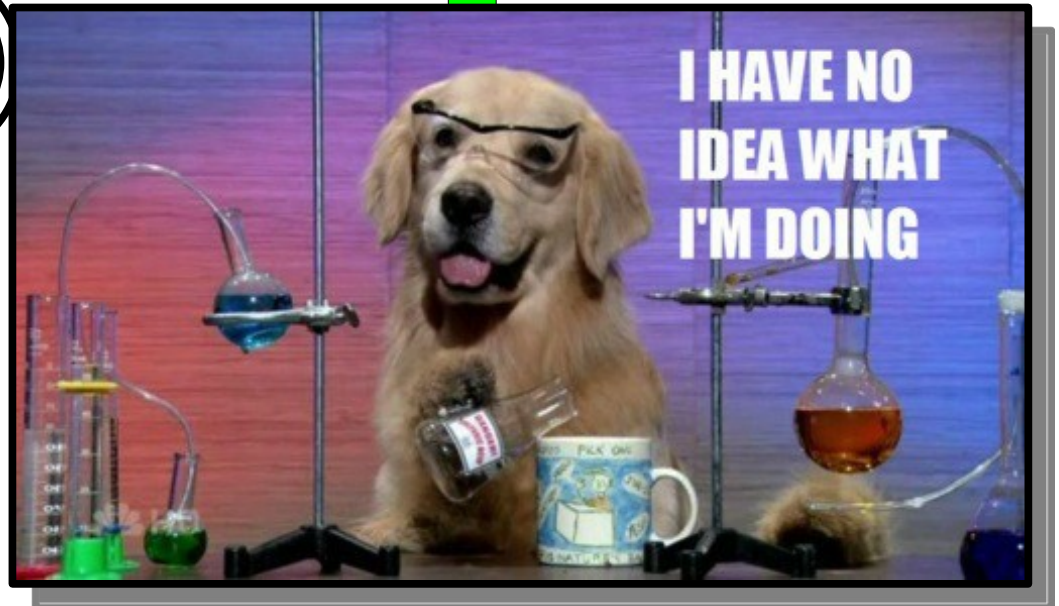
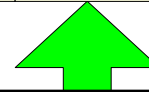
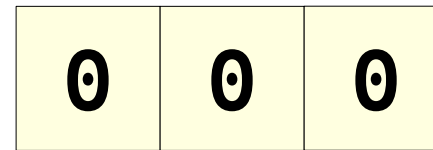
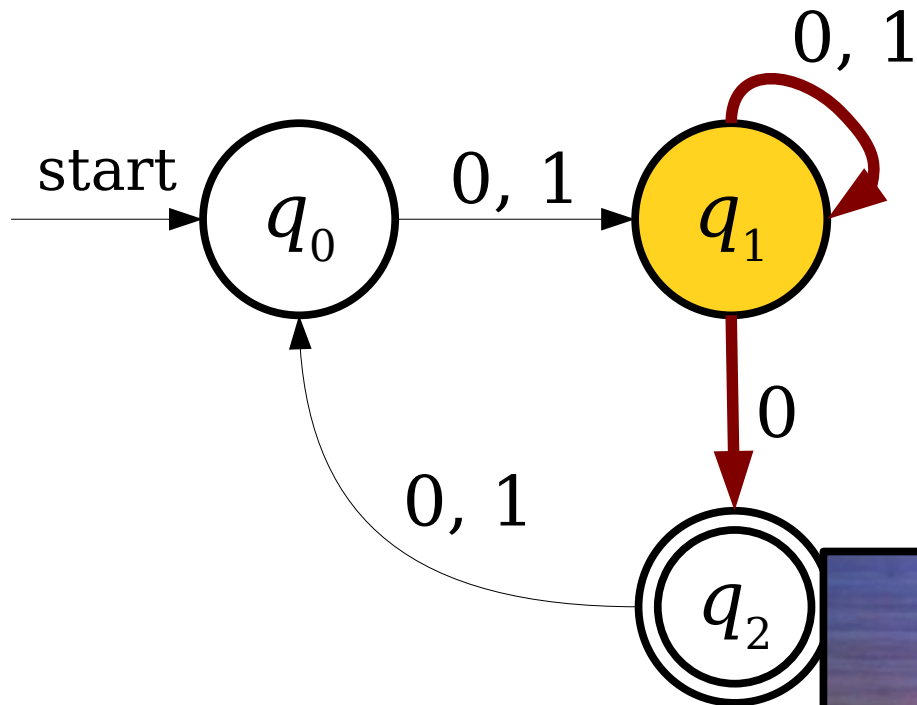
A Small Problem



0	1	1	0
---	---	---	---



Another Small Problem



The Need for Formalism

- In order to reason about the limits of what finite automata can and cannot do, we need to formally specify their behavior in *all* cases.
- All of the following need to be defined or disallowed:
 - What happens if there is no transition out of a state on some input?
 - What happens if there are *multiple* transitions out of a state on some input?

DFAs

- A **DFA** is a
 - **D**eterministic
 - **F**inite
 - **A**utomaton
- DFAs are the simplest type of automaton that we will see in this course.

DFA's

- A DFA is defined relative to some alphabet Σ .
- For each state in the DFA, there must be *exactly one* transition defined for each symbol in Σ .
 - This is the “deterministic” part of DFA.
- There is a unique start state.
- There are zero or more accepting states.